



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

SolariUM

Criação de um motor gráfico

Luís Ferreira
a98286

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Maio, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Ficheiros .obj	2
2.1.1. Motivação e Migração do Formato	2
2.1.2. Cálculo de Normais	3
2.1.3. Cálculo de Coordenadas de Textura e Mapeamento UV	7
3. Engine	12
3.1. Iluminação	12
3.2. Texturas	13
3.3. Shaders	14
3.3.1. Motivação	14
3.3.2. Introdução Teórica	14
3.3.3. Implementação	15
3.3.4. Sistema de Camadas de Textura	15
3.4. Mipmapping	17
4. Modelo do Sistema Solar	18
4.1. Novo Formato XML	18
4.2. Exemplos de XML Atualizado e Fontes de Texturas	18
4.2.1. Exemplos de XML	18
4.2.2. Fontes de Texturas	19
4.2.3. Nota sobre o Scatter	19
4.3. Resultados	20
5. Conclusão	23
Bibliografia	24
Lista de Siglas e Acrónimos	25
Anexos	26
Anexo 1 Logo da Universidade do Minho	26

Lista de Figuras

Figura 1	Representação da normal da esfera.	3
Figura 2	Representação da normal do cilindro.	4
Figura 3	Cálculo da normal do cone num ponto fixo.	5
Figura 4	Representação da normal do cone.	5
Figura 5	Representação da normal do toro.	6
Figura 6	Representação da normal das superfícies de <i>Bezier</i>	7
Figura 7	Representação do mapeamento das coordenadas de textura da esfera. . . .	8
Figura 8	Representação do mapeamento das coordenadas de textura da icoesfera. . 9	
Figura 9	Representação do mapeamento das coordenadas de textura do torus. . . .	9
Figura 10	Representação do mapeamento das coordenadas de textura do cone. . . .	10
Figura 11	Representação do mapeamento das coordenadas de textura do cubo. . . .	11
Figura 12	Teste iluminação.	20
Figura 13	Teste com textura de mapeamento.	20
Figura 14	Teste com textura de de tijolo.	20
Figura 15	Textura mapeamento.	21
Figura 16	Modelo detalhado da terra (visível componente especular e nuvens). . . .	21
Figura 17	Modelo detalhado da terra (visível componente noturna).	21
Figura 18	Modelo do sistema solar vista interna.	21
Figura 19	Modelo do sistema solar vista externa.	21
Figura 20	Modelo do sistema solar vista do Sol.	22
Figura 21	Modelo do sistema solar vista interna dos planetas próximos.	22

1. Introdução

O presente relatório pretende descrever o processo de desenvolvimento do trabalho laboratorial desenvolvido no âmbito da Unidade Curricular de Computação Gráfica.

Com base no enunciado fornecido o objetivo é criar um sistema completo de geração e visualização de cenas tridimensionais, composto por duas aplicações principais: o *generator*, responsável pela criação de modelos geométricos, e a *engine*, encarregue da leitura de ficheiros XML, carregamento de modelos e renderização das cenas utilizando a API *OpenGL* [1]. O trabalho é desenvolvido de forma incremental, estando dividido em quatro fases distintas, cada uma introduzindo novos conceitos fundamentais da Computação Gráfica.

Na 1ª fase, foi implementada a leitura do ficheiro XML no arranque da aplicação e criadas as estruturas de dados adequadas para armazenar a informação da cena em memória. O *generator* produz modelos geométricos básicos e a *engine* carrega-os e desenha-os.

Durante a 2ª fase, foi implementado o suporte a transformações geométricas hierárquicas (translação, rotação e escala), organizando a cena sob a forma de uma árvore de grupos. Aplicaram-se transformações aos modelos e seus subgrupos. O cenário de teste utilizado foi um modelo estático do sistema solar, que se usará para prosseguir com o projeto.

Na 3ª fase, o *generator* passou a suportar a geração de superfícies a partir de *patches* de Bezier. Na *engine*, implementaram-se transformações dependentes do tempo, nomeadamente translações ao longo de curvas Catmull-Rom e rotações contínuas. O suporte a VBOs para renderização foi também adicionado. O cenário demonstrativo passou a ser um modelo do sistema solar animado.

Na 4ª fase, devem ser geradas normais e coordenadas de textura no *generator*. Já na *engine* deve ser implementado o suporte a iluminação, materiais, texturas e diferentes tipos de luz definidos no XML. O resultado final deve ser o sistema solar animado com iluminação e texturas aplicadas.

2. Generator

2.1. Ficheiros .obj

2.1.1. Motivação e Migração do Formato

Nas fases anteriores, o *generator* produzia ficheiros no formato .3d — um formato simples e proprietário que armazenava apenas as coordenadas (x, y, z) de cada vértice dos triângulos gerados. Este formato era suficiente para a renderização de cor plana, mas revela-se inadequado para as novas funcionalidades introduzidas nesta fase.

Para suportar iluminação com o modelo de Phong, é necessário que cada vértice possua um vetor normal associado, que indica a orientação da superfície naquele ponto e é utilizado no cálculo da componente difusa e especular da iluminação. Para suportar texturas, é igualmente necessário que cada vértice possua coordenadas de textura (também designadas coordenadas UV), que mapeiam pontos do triângulo 3D para posições na imagem de textura 2D.

O formato .3d não suporta nenhum destes atributos adicionais. Assim, optou-se pela migração para o formato .obj (Wavefront OBJ) [2], um formato amplamente utilizado e padronizado na indústria que suporta nativamente a exportação de posições, normais e coordenadas de textura. A adoção deste formato elimina a necessidade de um formato proprietário e torna os modelos gerados compatíveis com ferramentas externas de visualização e edição 3D.

A estrutura de um ficheiro .obj gerado pelo sistema é a seguinte:

```
# Ficheiro gerado pelo generator
v  x y z          # posição do vértice
vn nx ny nz       # normal do vértice
vt  u v           # coordenada de textura
f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3 # face triangular
```

Cada face referencia vértices pelo seu índice na lista, seguido do índice de textura e do índice de normal, separados pelo caractere “/”. A *engine* foi atualizada de forma correspondente para carregar este formato e popular os *buffers* de normais e coordenadas de textura nos VBOs.

A linha de comandos do generator foi também estendida para produzir ficheiros .obj:

```
./generator sphere <radius> <slices> <stacks> output.obj
./generator box <length> <divisions> output.obj
./generator cone <radius> <height> <slices> <stacks> output.obj
./generator plane <length> <divisions> output.obj
./generator bezier <file.patch> <tessellations> output.obj
```

Por uma questão de retrocompatibilidade o sistema continua a permitir a criação de .3d, diferenciando apenas pela extensão colocada no *input* da chamada ao *generator*.

2.1.2. Cálculo de Normais

As normais são vetores unitários perpendiculares à superfície em cada ponto, essenciais para o modelo de iluminação de Phong. A abordagem de cálculo varia consoante a primitiva geométrica.

Esferas

Para uma esfera de raio r , a implementação usa coordenadas esféricas com $\varphi \in [0, \pi]$ como ângulo de colatitude (medido a partir do polo norte, ao longo das *stacks*) e $\theta \in [0, 2\pi]$ como ângulo de longitude (ao longo dos *slices*). O ponto na superfície é:

$$P(\varphi, \theta) = (r \sin(\varphi) \cos(\theta), r \cos(\varphi), r \sin(\varphi) \sin(\theta))$$

A normal analítica coincide com a posição normalizada, uma vez que todos os pontos estão à distância r da origem:

$$\hat{n}(\varphi, \theta) = (\sin(\varphi) \cos(\theta), \cos(\varphi), \sin(\varphi) \sin(\theta))$$

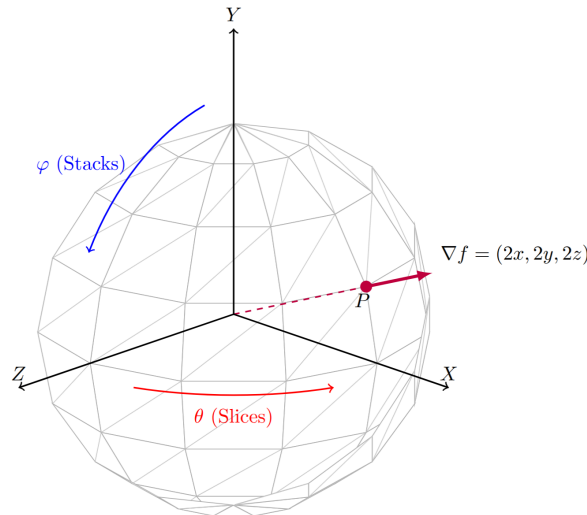


Figura 1: Representação da normal da esfera.

Esta propriedade decorre do facto de a esfera ser uma superfície de nível da função $f(x, y, z) = x^2 + y^2 + z^2 - r^2$, cujo gradiente $\nabla f = (2x, 2y, 2z)$ aponta radialmente para fora. Como a parametrização por stacks e slices conhece φ e θ explicitamente em cada vértice, a normal analítica está disponível diretamente, sem necessidade de qualquer produto vetorial.

No caso da esfera gerada através de um icosaedro, a propriedade radial da continua válida:

$$\hat{n} = \frac{P}{r}$$

A diferença prática face à esfera UV é que, na icosfera, a normal de cada vértice é calculada normalizando a sua posição após a projeção para a esfera, em vez de ser deduzida analiticamente dos ângulos θ e φ . O resultado matemático é equivalente (ambas produzem a normal radial exata), mas a abordagem computacional é distinta: a esfera UV calcula a normal a partir dos ângulos da parametrização, enquanto a icosfera a obtém a partir da posição do vértice já projetado.

Cilindro e Cone

Para a superfície lateral de um cilindro, a normal em cada ponto aponta radialmente para fora no plano xOz . Para um ponto na superfície com ângulo de slice θ , a normal lateral é diretamente:

$$\hat{n}_{\text{lateral}} = (\cos(\theta), 0, \sin(\theta))$$

Para as tampas, as normais são constantes e apontam segundo o eixo y :

- Tampa inferior ($y = 0$): $\hat{n} = (0, -1, 0)$
- Tampa superior ($y = h$): $\hat{n} = (0, 1, 0)$

No entanto, o código usa a mesma normal lateral também para os vértices da aresta da tampa — ou seja, os vértices da circunferência das tampas recebem a normal radial $(\cos(\theta), 0, \sin(\theta))$, e apenas o vértice central recebe a normal axial $(0, \pm 1, 0)$. Esta escolha produz uma transição mais suave entre a tampa e a superfície lateral ao nível do *shading*.

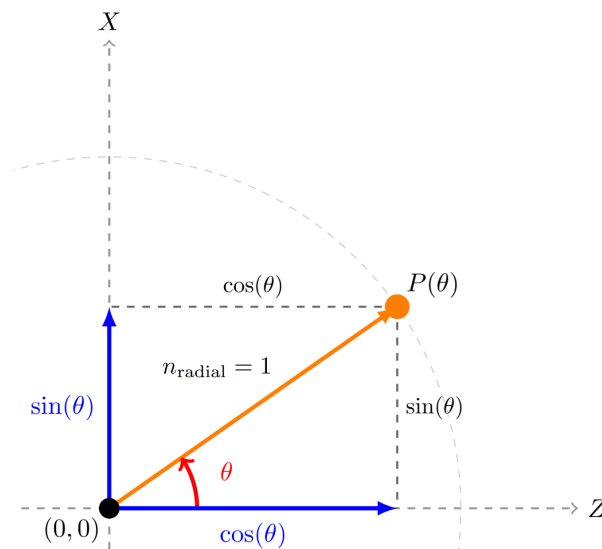


Figura 2: Representação da normal do cilindro.

No caso do cone, a normal lateral é calculada analiticamente a partir da inclinação da superfície. O vetor normal a um cone de raio r e altura h é perpendicular à geratriz.

Definindo $L = \sqrt{h^2 + r^2}$ como o comprimento da geratriz, a normal lateral no ângulo de slice θ é:

$$\hat{n}_{\text{lateral}} = \left(\frac{h}{L} \cos(\theta), \frac{r}{L}, \frac{h}{L} \sin(\theta) \right)$$

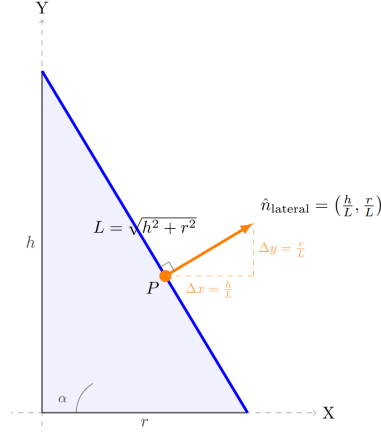


Figura 3: Cálculo da normal do cone num ponto fixo.

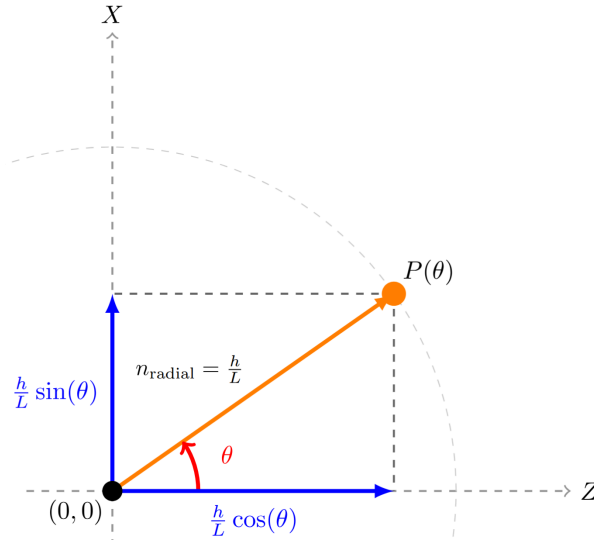


Figura 4: Representação da normal do cone.

onde $\frac{h}{L}$ e $\frac{r}{L}$ são os cossenos dos ângulos da normal com os eixos horizontal e vertical, respetivamente. Para o vértice do apex, a normal é calculada como a média das normais laterais vizinhas, avaliada no ângulo médio do slice. Os vértices da aresta da base usam a mesma normal lateral radial, e o vértice central da base usa $(0, -1, 0)$.

Caixa (Box)

Para uma caixa, cada face tem uma normal constante perpendicular à face. As seis normais possíveis são os vetores: $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, $(0, 0, \pm 1)$, consoante a face.

Plano

Um plano horizontal no eixo xOz tem normal constante:

$$\hat{n} = (0, 1, 0)$$

Toro

Um toro de raio maior R (distância do centro do toro ao centro do tubo) e raio menor r (raio do próprio tubo) é parametrizado pelos ângulos $\theta \in [0, 2\pi]$ (ângulo em torno do eixo central, no plano xOz) e $\varphi \in [0, 2\pi]$ (ângulo em torno do tubo). O ponto na superfície é:

$$P(\theta, \varphi) = ((R + r \cos(\varphi)) \cos(\theta), r \sin(\varphi), (R + r \cos(\varphi)) \sin(\theta))$$

Para calcular a normal, obtém-se o centro do tubo para o ângulo θ atual — o ponto $C(\theta)$ no círculo central do toro:

$$C(\theta) = (R \cos(\theta), 0, R \sin(\theta))$$

A normal em $P(\theta, \varphi)$ é o vetor que aponta do centro do tubo $C(\theta)$ para o ponto P , normalizado:

$$\hat{n}(\theta, \varphi) = \text{normalize}(P(\theta, \varphi) - C(\theta)) = (\cos(\varphi) \cos(\theta), \sin(\varphi), \cos(\varphi) \sin(\theta))$$

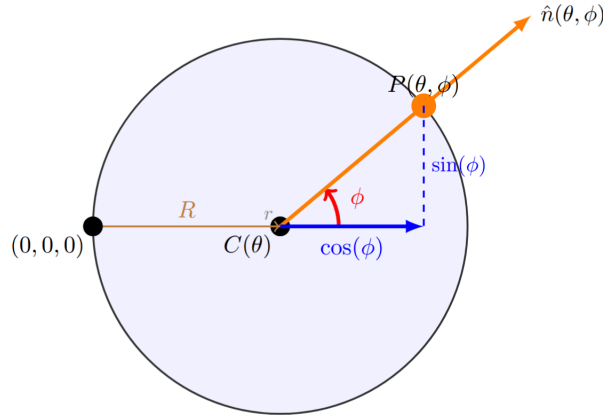


Figura 5: Representação da normal do toro.

Geometricamente, isto significa que a normal aponta radialmente para fora do tubo em cada ponto, de forma análoga à normal radial da esfera, mas agora em relação ao eixo local do tubo e não à origem global. As derivadas parciais confirmam este resultado:

$$\frac{\partial P}{\partial \theta} = (-(R + r \cos \varphi) \sin \theta, 0, (R + r \cos \varphi) \cos \theta)$$

$$\frac{\partial P}{\partial \varphi} = (-r \sin(\varphi) \cos(\theta), r \cos(\varphi), -r \sin(\varphi) \sin(\theta))$$

O produto vetorial $\partial P / \partial \varphi \times \partial P / \partial \theta$, normalizado, coincide com a expressão analítica da normal acima.

Superfícies de Bézier

Para patches de Bézier, a normal em cada ponto é calculada como o produto vetorial das derivadas parciais da superfície em u e v :

$$\hat{n}(u, v) = \text{normalize}\left(\frac{\partial Q}{\partial u} \times \frac{\partial Q}{\partial v}\right)$$

onde as derivadas parciais são obtidas pela diferenciação da fórmula matricial de Bézier:

$$\frac{\partial Q}{\partial u} = \sum_{i=0}^3 \sum_{j=0}^3 B'_{i(u)} B_{j(v)} P_{i,j}$$

$$\frac{\partial Q}{\partial v} = \sum_{i=0}^3 \sum_{j=0}^3 B_{i(u)} B'_{j(v)} P_{i,j}$$

com $B'_{k(t)}$ as derivadas dos polinómios de Bernstein:

$$B'_0(t) = -3(1-t)^2, \quad B'_1(t) = 3(1-t)(1-3t), \quad B'_2(t) = 3t(2-3t), \quad B'_3(t) = 3t^2$$

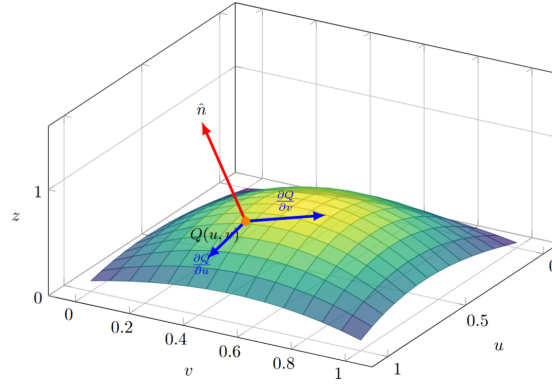


Figura 6: Representação da normal das superfícies de *Bezier*.

2.1.3. Cálculo de Coordenadas de Textura e Mapeamento UV

As coordenadas de textura $(u, v) \in [0, 1]^2$ mapeiam cada ponto da superfície 3D para uma posição na imagem de textura 2D. O processo de calcular estas coordenadas designa-se mapeamento UV. A abordagem varia consoante a primitiva.

Esfera UV — Mapeamento Esférico Paramétrico

Para a esfera parametrizada por stacks (colatitude $\varphi \in [0, \pi]$) e slices (longitude $\theta \in [0, 2\pi]$), o mapeamento UV é direto a partir dos índices de geração:

$$u = 1 - j/\text{slices}, \quad v = 1 - i/\text{stacks}$$

onde i é o índice do stack e j o índice do slice. Equivalentemente em ângulos: $u = 1 - \theta/2\pi$ e $v = 1 - \varphi/\pi$. O valor $v = 1$ corresponde ao polo norte ($\varphi = 0$) e $v = 0$ ao polo sul ($\varphi = \pi$), de modo a que mapas de textura equiretangulares fiquem orientados corretamente (polo norte no topo da imagem).

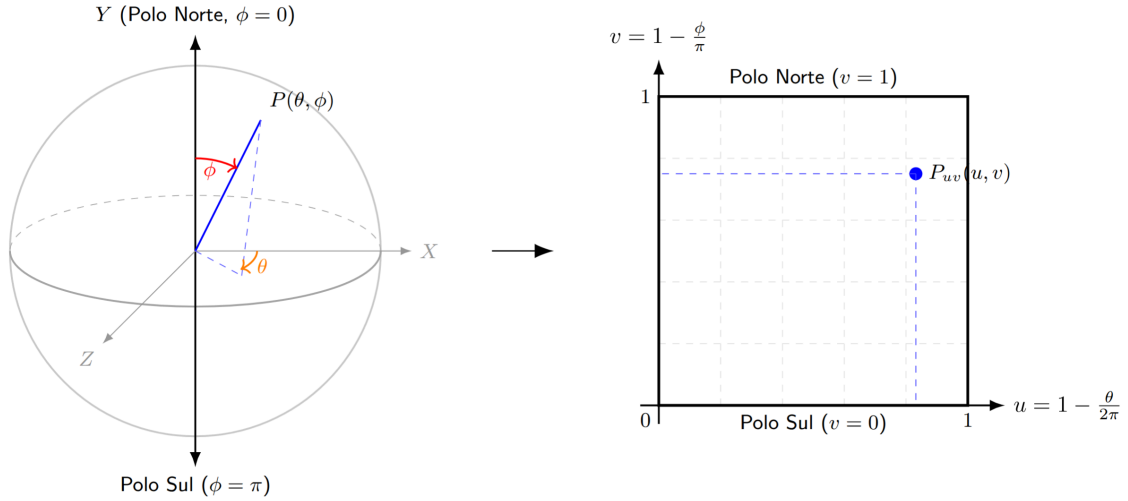


Figura 7: Representação do mapeamento das coordenadas de textura da esfera.

O espelhamento horizontal ($u = 1 - j/\text{slices}$ em vez de $u = j/\text{slices}$) é necessário para garantir a coerência da orientação da textura em todas as figuras que usam coordenadas esféricas ou $\arctan2$. Sem esta inversão, a textura fica espelhada horizontalmente face ao esperado para mapas equiretangulares *standard* — o hemisfério leste aparece no lado oeste e vice-versa. Esta convenção é adotada de forma consistente na esfera UV, na icosfera, no toro, no cilindro e no cone.

Para os triângulos do polo norte e do polo sul, o vértice do topo recebe $u = (u_1 + u_2)/2$ — a média dos u dos dois vértices do lado oposto do triângulo — evitando assim a degenerescência UV no polo. Não são criados vértices duplicados explícitos na costura, a continuidade da textura depende da configuração de `GL_REPEAT`.

Icosfera — Mapeamento Esférico por Projeção

A icosfera não possui uma parametrização (φ, θ) explícita por vértice, pelo que o mapeamento UV é calculado a partir da posição de cada vértice já projetado para a esfera. Para um vértice $P = (x, y, z)$ com $|P| = r$, as coordenadas esféricas são recuperadas por:

$$u = 1 - \frac{\text{atan2}(z, x) + \pi}{2\pi}, \quad v = 1 - \frac{\arccos(\frac{y}{r})}{\pi}$$

A função $\text{atan2}(z, x)$ devolve a longitude no intervalo $]-\pi, \pi]$, que é deslocada para $[0, 1]$ somando π antes de normalizar. A inversão aplica o mesmo espelhamento horizontal da esfera UV, garantindo coerência de orientação entre as duas primitivas. O valor $v = 1 - \frac{\arccos(\frac{y}{r})}{\pi}$ coloca o polo norte ($y = r$) em $v = 1$ e o polo sul ($y = -r$) em $v = 0$, com a mesma orientação da esfera UV.

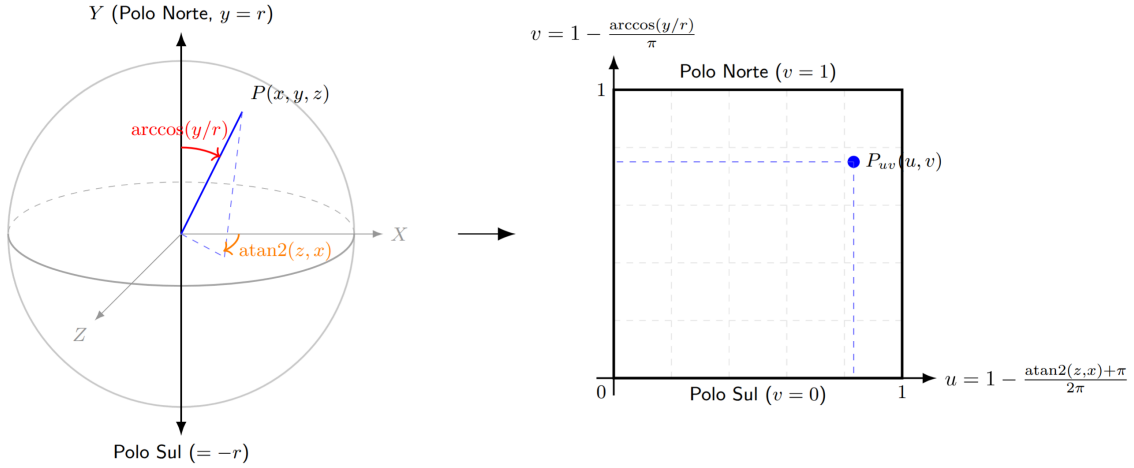


Figura 8: Representação do mapeamento das coordenadas de textura da icoesfera.

O problema de costura nos triângulos que cruzam o meridiano $\text{atan2} = \pm\pi$ é resolvido com duas correções aplicadas a cada triângulo após calcular os UV dos três vértices:

- Costura lateral: determina-se o u máximo dos três vértices ($u_{\max}$). Qualquer vértice com $u_{\max} - u > 0.5$ recebe $u + 1$, fazendo com que todos os u do triângulo fiquem no mesmo “lado” do meridiano e a interpolação seja coerente.
- Polo: qualquer vértice cujo $|y|$ se aproxime de r (polo) recebe $u = (u_a + u_b)/2$, onde u_a e u_b são os u dos outros dois vértices do triângulo, evitando o colapso da textura no polo.

Toro — Mapeamento Toroidal

Para o toro parametrizado pelos ângulos θ e φ , o mapeamento UV mapeia cada ângulo diretamente para uma coordenada de textura:

$$u = 1 - i/\text{slices}, \quad v = j/\text{stacks}$$

onde $i \in \{0, \dots, \text{slices}\}$ é o índice do slice (ângulo θ) e $j \in \{0, \dots, \text{stacks}\}$ é o índice do stack (ângulo φ). Equivalentemente, $u = 1 - \theta/2\pi$ e $v = \varphi/2\pi$. A coordenada u percorre a textura ao longo da circunferência maior do toro (com espelhamento horizontal, por coerência com as restantes figuras esféricas) e v percorre-a ao longo do tubo.

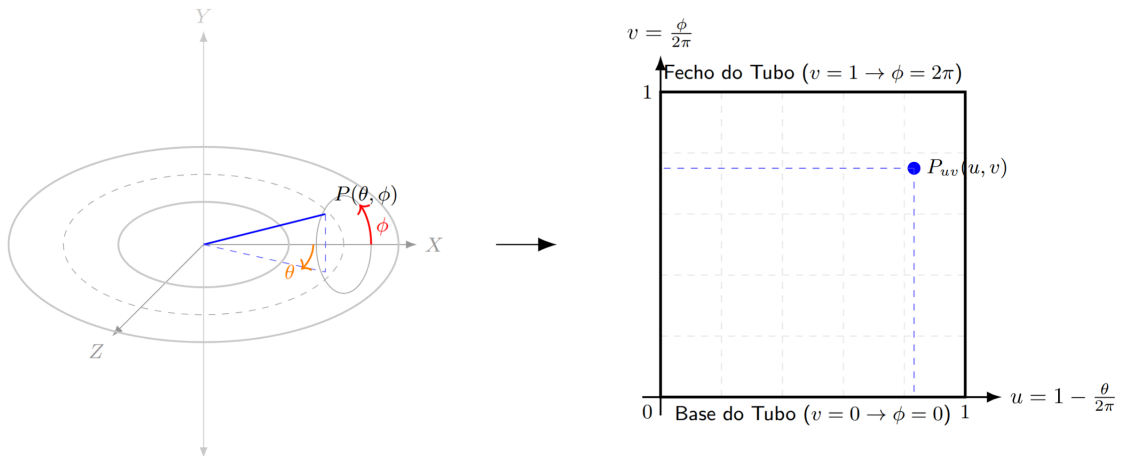


Figura 9: Representação do mapeamento das coordenadas de textura do torus.

A costura nas coordenadas $\theta = 2\pi$ e $\varphi = 2\pi$ é tratada implicitamente pela geração de quads: quando $i = \text{slices} - 1$ e $j = \text{stacks} - 1$, os vértices do quad seguinte usam os índices $i + 1$ e $j + 1$ que produzem $u = 1$ e $v = 1$, garantindo que o último quad fecha a superfície sem descontinuidade geométrica. Não são criados vértices duplicados explícitos — a continuidade visual depende da repetição de textura GL_REPEAT.

Cilindro — Mapeamento Cilíndrico

Para a superfície lateral de um cilindro, o mapeamento usa os índices de geração diretamente:

$$u = 1 - j/\text{slices}, \quad v = i/\text{stacks}$$

onde j é o índice do slice (ângulo $\theta \in [0, 2\pi]$) e i o índice do stack (altura $y \in [0, h]$). Equivalentemente, $u = 1 - \theta/2\pi$ e $v = y/h$.

Para as tampas, o mapeamento não usa coordenadas polares planas — em vez disso, cada vértice da aresta da tampa recebe $u = 1 - j/\text{slices}$ (o mesmo u angular invertido da superfície lateral) e $v = 0$ na tampa inferior ou $v = 1$ na tampa superior. O vértice central de cada tampa recebe $u = (u_1 + u_2)/2$ (média dos u dos dois vértices da aresta do triângulo corrente) e $v = 0$ ou $v = 1$. Esta abordagem produz um mapeamento em “leque” a partir do centro, análoga ao tratamento dos polos na esfera.

Cone — Mapeamento Cônico

Para a superfície lateral do cone, o mapeamento usa os mesmos índices angulares do cilindro:

$$u = 1 - j/\text{slices}, \quad v = i/\text{stacks}$$

onde j é o índice do slice e i o índice do stack (altura normalizada $y/h \in [0, 1]$). O vértice mais acima de cada triângulo no último stack recebe $u = (u_1 + u_2)/2$ e $v = 1$.

Para a base do cone, o mapeamento é idêntico ao das tampas do cilindro: os vértices da aresta recebem $u = 1 - j/\text{slices}$ e $v = 0$, e o vértice central recebe a média dos u e $v = 0$.

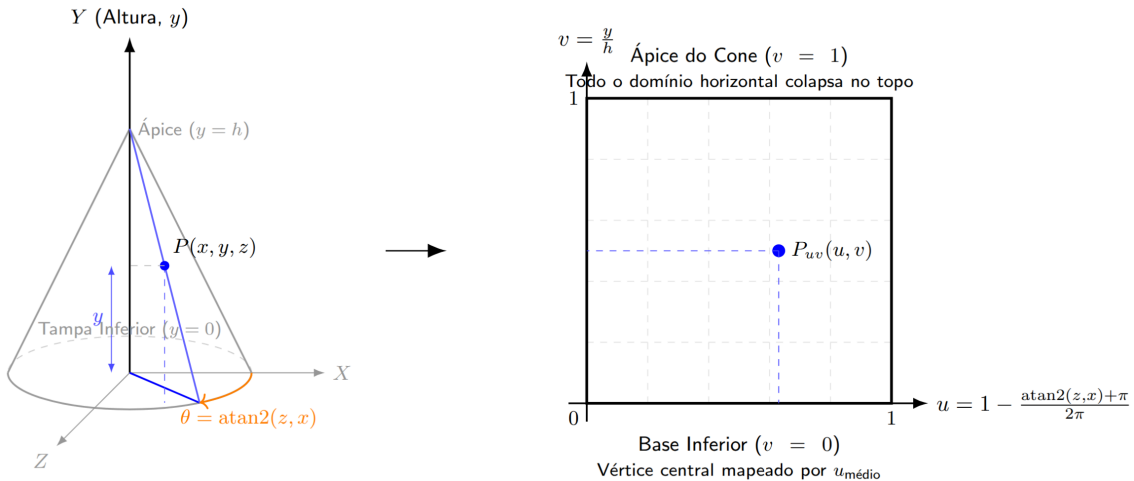


Figura 10: Representação do mapeamento das coordenadas de textura do cone.

Caixa — Mapeamento Cúbico

Para cada face da caixa, as coordenadas UV são calculadas a partir dos índices de subdivisão da face. Para a face com normal $(0, 1, 0)$ (face de cima), com i e j a variar de 0 a $d - 1$ (número de divisões):

$$u = i/d, \quad v = 1 - j/d$$

onde o espelhamento de v ($1 - j/d$) garante que a textura fica orientada corretamente na face superior. As restantes faces seguem o mesmo esquema de indexação, com as coordenadas locais da face a determinar quais os índices usados, e algumas faces com u espelhado ($1 - i/d$) para manter a coerência de orientação visual entre faces adjacentes.

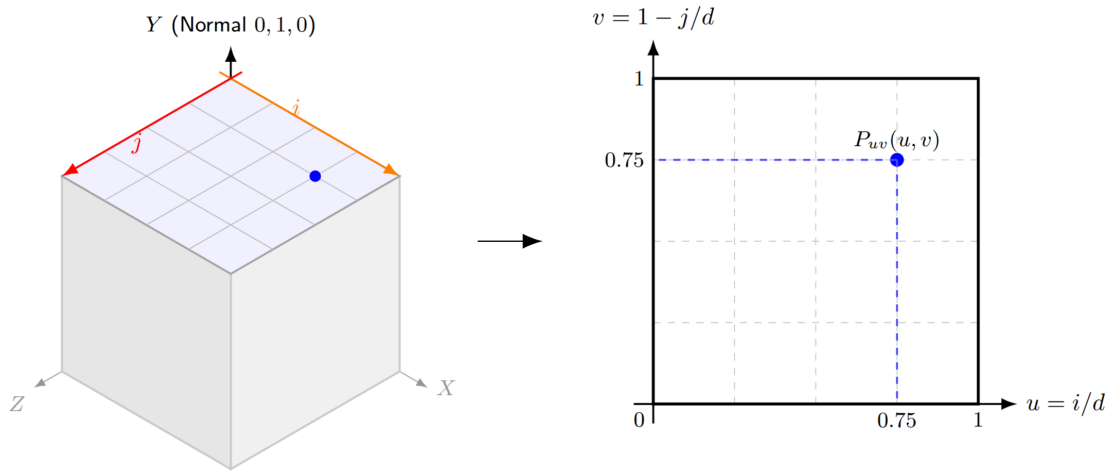


Figura 11: Representação do mapeamento das coordenadas de textura do cubo.

Plano — Mapeamento Planar

Para um plano de dimensão $L \times L$ no plano xOz , com d divisões, o mapeamento usa os índices diretamente:

$$u = i/d, \quad v = j/d$$

onde i e j são os índices de subdivisão ao longo de x e z respetivamente. Equivalentemente, $u = (x + L/2)/L$ e $v = (z + L/2)/L$, o que mapeia o canto $(-L/2, -L/2)$ para $(0, 0)$ e o canto $(L/2, L/2)$ para $(1, 1)$. A textura é aplicada uma única vez sobre toda a superfície do plano, independentemente do número de divisões.

Superfícies de Bézier — Mapeamento Paramétrico

Para patches de Bézier, o mapeamento UV é simplesmente os próprios parâmetros de avaliação da superfície:

$$u = u_{\text{param}}, \quad v = v_{\text{param}}$$

onde $(u_{\text{param}}, v_{\text{param}}) \in [0, 1]^2$ são os parâmetros de tesselação. Esta abordagem garante um mapeamento contínuo e sem distorção nos parâmetros da superfície, embora possa introduzir distorção espacial em superfícies com forte curvatura.

3. Engine

3.1. Iluminação

A iluminação foi implementada com base no modelo de Phong [3], que descreve a interação da luz com uma superfície através de três componentes: ambiente, difusa e especular. O modelo é avaliado por fragmento (per-fragment lighting), o que produz resultados visuais de maior qualidade do que a alternativa por vértice (Gouraud shading).

A equação de iluminação de Phong para uma fonte de luz l é:

$$I = k_a \cdot I_a + k_d \cdot (\hat{n} \cdot \hat{l}) \cdot I_d + k_s \cdot (\hat{r} \cdot \hat{v})^\alpha \cdot I_s$$

onde:

- k_a, k_d, k_s são os coeficientes ambiente, difuso e especular do material;
- I_a, I_d, I_s são as intensidades ambiente, difusa e especular da fonte de luz;
- $\hat{n}$ é a normal unitária da superfície no fragmento;
- $\hat{l}$ é o vetor unitário que aponta do fragmento para a fonte de luz;
- $\hat{r}$ é o vetor de reflexão de $\hat{l}$ em relação a $\hat{n}$, dado por $\hat{r} = 2(\hat{n} \cdot \hat{l})\hat{n} - \hat{l}$;
- $\hat{v}$ é o vetor unitário que aponta do fragmento para a câmara;
- α é o expoente de brilho (shininess) do material.

O formato XML foi estendido para suportar a definição de fontes de luz e propriedades de material. As fontes de luz são declaradas sob uma tag `<lights>` e os materiais sob a tag `<color>` dentro de cada modelo:

```
<lights>
  <light type="point" posX="0" posY="0" posZ="0" />
  <light type="directional" dirX="1" dirY="0" dirZ="0" />
  <light type="spot" posX="0" posY="10" posZ="0"
    dirX="0" dirY="-1" dirZ="0" cutoff="30" />
</lights>
```

```
<model file="earth.obj">
  <color>
    <ambient R="0.2" G="0.2" B="0.2" A="1.0" />
    <diffuse R="0.8" G="0.8" B="0.8" A="1.0" />
    <specular R="0.5" G="0.5" B="0.5" A="1.0" />
    <emissive R="0.0" G="0.0" B="0.0" A="1.0" />
    <shininess value="64" />
  </color>
</model>
```

```
</color>  
</model>
```

São suportados três tipos de fontes de luz:

- Pontual (*point*): emite luz em todas as direções a partir de uma posição no espaço. A atenuação é calculada em função da distância.
- Direcional (*directional*): simula fontes de luz muito distantes (como o Sol), com raios paralelos e sem atenuação.
- Spot: emite um cone de luz a partir de uma posição, com um ângulo de abertura (*cutoff*) definido.

Na engine, as propriedades de cada fonte de luz são passadas ao shader via uniforms, e o número de luzes ativas é também transmitido para que o shader itere apenas sobre as fontes relevantes.

3.2. Texturas

O suporte a texturas foi implementado utilizando a biblioteca “*stb_image*” para carregamento de imagens nos formatos PNG, JPEG e outros formatos comuns. Cada textura é carregada uma única vez e armazenada numa *cache* indexada pelo nome do ficheiro, evitando leituras repetidas do disco e duplicação de objetos *OpenGL*.

O processo de carregamento de uma textura começa por verificar se a textura já existe em *cache*. Se o ficheiro já tiver sido carregado anteriormente, a função devolve imediatamente o identificador da textura armazenado na *cache*, evitando carregar e processar os dados novamente.

Caso a textura ainda não esteja em *cache*, a imagem é carregada do disco usando *stbi_load*, que devolve os dados dos píxeis juntamente com a largura, altura e número de canais da imagem.

Depois disso, é criado um novo objeto de textura *OpenGL* através de *glGenTextures*. A textura é então associada ao alvo *GL_TEXTURE_2D* usando *glBindTexture*, tornando-a a textura atualmente ativa para configuração.

Os dados dos píxeis são enviados para a GPU com *glTexImage2D*, que cria efetivamente a textura na memória gráfica. Em seguida, são gerados os mipmap com *glGenerateMipmap*, permitindo diferentes níveis de detalhe para melhorar qualidade visual e desempenho em diferentes distâncias.

Após o upload, são configurados os parâmetros da textura, incluindo os filtros de minificação/magnificação e os modos de wrapping.

Finalmente, o identificador da textura é armazenado em *textureCache* associado ao nome do ficheiro, para reutilização futura, e a função devolve o identificador da textura carregada.

As texturas são declaradas no XML dentro da tag *<texture>* de cada modelo:

```
<model file="earth.obj">
  <texture file="earth.jpg" role="diffuse" blend="multiply" />
</model>
```

Na renderização, a textura é ativada na unidade de textura correspondente antes de cada draw call. O fragment shader recebe o identificador da unidade de textura e amostra a cor com a função `texture(sampler, texcoord)` do GLSL.

3.3. Shaders

3.3.1. Motivação

O modelo de iluminação de Phong e o suporte a texturas simples seriam, em princípio, implementáveis com o *pipeline* fixo do OpenGL através das funções `glLightfv`, `glMaterialfv` e `glTexEnv`. No entanto, uma vontade nossa — representar a Terra com camadas múltiplas (textura de dia, textura de noite, mapa de nuvens e mapa especular) — exige um controlo preciso sobre a combinação das contribuições de cada textura no valor final do fragmento, algo que o pipeline fixo não permite de forma flexível.

Por exemplo, a textura de noite da Terra (emissão de luz artificial) deve ser visível apenas nos fragmentos que estão no lado escuro do planeta (onde $\hat{n} \cdot \hat{l} \leq 0$), e a sua intensidade deve transitar suavemente com a contribuição difusa. Esta lógica condicional na combinação de texturas é impossível de expressar sem shaders programáveis.

Adicionalmente, os shaders permitem implementar a iluminação *per-fragment* de forma mais eficiente e generalizada, suportando múltiplas fontes de luz sem os limites impostos pelo pipeline fixo do OpenGL.

3.3.2. Introdução Teórica

Um shader é um programa executado diretamente na GPU, em paralelo, para cada vértice (*vertex shader*) ou para cada fragmento (*fragment shader*) produzido pela rasterização. Em OpenGL, os shaders são escritos em GLSL (OpenGL Shading Language), uma linguagem de alto nível semelhante ao C.

O pipeline programável de OpenGL funciona da seguinte forma:

1. Vertex Shader: executado uma vez por vértice. Recebe os atributos do vértice (posição, normal, coordenada de textura) e produz a posição em espaço de clip e os varyings interpolados para o fragment shader.
2. Rasterização: a GPU interpola os varyings entre os vértices de cada triângulo para cada fragmento coberto.
3. Fragment Shader: executado uma vez por fragmento. Recebe os varyings interpolados e produz a cor final do fragmento.

A comunicação com os shaders é feita através de:

- Uniforms: variáveis globais passadas pela CPU, iguais para todos os vértices/fragmentos de um draw call (ex.: matrizes de transformação, propriedades de luz, samplers de textura).
- Attributes: atributos por vértice (posição, normal, UV), fornecidos pelos VBOs.
- Varyings: valores calculados no vertex shader e interpolados para o fragment shader (ex.: normais, coordenadas de textura, posição no espaço da câmara).

3.3.3. Implementação

O sistema de shaders foi implementado através da classe singleton `ShaderManager` (`shader_manager.h` / `shader_manager.cpp`), que gera e compila programas GLSL dinamicamente em tempo de execução, em função das camadas de textura declaradas para cada modelo.

Em vez de ficheiros GLSL estáticos em disco, os shaders são construídos programaticamente: o vertex shader é fixo e compilado uma única vez para todos os modelos, enquanto o fragment shader é gerado pela função `buildFragmentSource()`, que itera sobre a lista de `TextureLayer` do modelo e emite código GLSL para cada camada — respeitando os atributos `blend`, `mixFactor` e `useChannel` de cada uma. Os modos de composição suportados são `multiply`, `add`, `overlay` e `mix/replace`.

Os programas compilados são guardados numa cache indexada por uma chave estrutural derivada dos atributos `blend:mixFactor:useChannel` de cada camada (excluindo `opacity`, que é passada como uniform por frame). Assim, dois modelos com a mesma estrutura de camadas reutilizam o mesmo programa GLSL.

A seleção e ativação do shader é feita via `ShaderManager::bind()`: qualquer modelo com pelo menos uma `TextureLayer` é renderizado com o shader dinâmico; modelos sem texturas utilizam o pipeline fixo do OpenGL com iluminação de Phong via `glMaterialfv`.

3.3.4. Sistema de Camadas de Textura

A implementação não usa shaders GLSL pré-escritos em ficheiro — o fragment shader é gerado em C++ em tempo de execução, na função `buildFragmentSource()`, a partir da lista de `TextureLayer` de cada modelo. Isto permite que qualquer modelo declare qualquer combinação de texturas no XML, sem necessidade de escrever um novo *shader* manualmente.

Cada `<texture>` no XML define uma camada com quatro campos que controlam o seu comportamento:

- `role` — etiqueta livre para o humano (ex.: `"diffuse"`, `"night"`). O engine nunca a lê para tomar decisões; serve apenas de documentação.
- `useChannel` — que canal da imagem usar: `"rgb"` (omissão) usa as três componentes de cor; `"r"`, `"g"`, `"b"` ou `"a"` extrai um único canal como escala de cinzento. Útil quando a textura codifica informação num canal específico (ex.: mapa de especularidade em tons de cinzento no canal `r`).
- `mixFactor` — peso dinâmico da camada, calculado por fragmento:

Valor	Significado
"1.0"	sempre 100% (omissão)
"ndI"	$\hat{n} \cdot \hat{l}$ — mais forte onde a luz incide diretamente
"1-ndI"	inverso — mais forte nas zonas de sombra
"specular"	proporcional ao brilho especular calculado
float literal	ex.: "0.5" — peso fixo

- blend — como a camada se combina com a cor acumulada até ao momento. O fragment *shader* processa as camadas sequencialmente; em cada passo, color é o resultado de todas as camadas anteriores:

Modo	Fórmula	Efeito
"mix" / "replace"	<code>mix(color, texSample, w)</code>	substitui suavemente a cor acumulada
"multiply" (omissão)	<code>mix(color, color × texSample, w)</code>	escurece — bom para detalhes de superfície
"add"	<code>color += texSample × w</code>	soma brilho — bom para emissivos e luzes
"overlay"	mistura híbrida claro/escuro	aumenta contraste sem saturar

O peso efetivo de cada camada é sempre $w = \text{mixFactor} \times \text{opacity}$. A *opacity* é o único campo que pode mudar a cada frame (por isso não entra na chave de cache do *shader*).

Como exemplo prático, a configuração da Terra no XML da fase 4 declara quatro camadas:

```
<model file="icosphere.obj" shader="earth_daynight">
  <texture role="diffuse" file="textures/2k_earth_daymap.jpg"
    blend="multiply" opacity="1.0" />
  <texture role="night" file="textures/2k_earth_nightmap.jpg"
    blend="mix" opacity="0.9" mixFactor="1-ndI" />
  <texture role="clouds" file="textures/2k_earth_clouds.jpg"
    blend="overlay" opacity="0.4" />
  <texture role="specular" file="textures/2k_earth_specular_map.jpg"
    blend="add" opacity="1.0" useChannel="r"
  mixFactor="specular" />
</model>
```

O fragment *shader* gerado para esta combinação calcula, por fragmento: (1) a cor de Phong base; (2) mistura a textura de dia; (3) adiciona as luzes noturnas ponderadas por $1 - \hat{n} \cdot \hat{l}$; (4) aplica o mapa especular para distinguir superfícies brilhantes (oceanos) das matte (continentes).

As uniforms de iluminação (`uLightPosEye`, `uGlobalAmbient`) são atualizadas a cada *frame* via `updateLightPosition()` e `bind()`, antes de cada *draw call*.

3.4. Mipmapping

O mipmapping é uma técnica de filtragem de texturas que consiste em pré-calcular versões da textura a resoluções progressivamente menores (cada nível tem metade da largura e altura do anterior). Quando um fragmento mapeia para uma área grande da textura (i.e., a textura está a ser renderizada a uma distância grande ou em perspectiva), o OpenGL seleciona automaticamente o nível de mipmap mais adequado, evitando aliasing e o efeito de “cintilação” que ocorreria ao amostrar uma textura de alta resolução com muito poucos fragmentos.

A ativação do mipmapping é feita imediatamente após o carregamento de cada textura com `glGenerateMipmap(GL_TEXTURE_2D)`.

O filtro `GL_LINEAR_MIPMAP_LINEAR` (trilinear filtering) interpola linearmente entre os dois níveis de mipmap mais próximos e entre os texels de cada nível, produzindo transições suaves entre níveis e uma qualidade visual superior à filtragem bilinear simples. O filtro de magnificação `GL_LINEAR` é utilizado quando a textura é ampliada (fragmento menor que um texel).

Todas as texturas do projeto têm mipmapping ativado por omissão, incluindo as texturas dos planetas, nuvens e mapas especulares.

4. Modelo do Sistema Solar

4.1. Novo Formato XML

O ficheiro XML do sistema solar foi significativamente estendido para suportar os novos elementos introduzidos nesta fase: fontes de luz, materiais, texturas e shaders.

A estrutura geral do XML foi reorganizada para incluir uma secção `<lights>` ao nível do elemento raiz `<world>`, e as tags `<model>` foram enriquecidas com elementos filhos `<color>` e `<texture>`.

O atributo `shader` indica qual o programa GLSL a utilizar na renderização do modelo. Para a maioria dos corpos celestes, utiliza-se o shader padrão (omitido ou `shader="default"`). Para a Terra, o shader especializado `earth` é indicado explicitamente, activando a combinação das quatro camadas de textura.

O atributo `blend` foi introduzido para modelos como as nuvens da Terra, onde se deseja uma mistura baseada no canal alfa da textura de nuvens com o modelo de iluminação subjacente.

A fonte de luz principal da cena é o Sol, modelado como uma fonte pontual colocada na origem. Adicionalmente, foi adicionada uma componente ambiente global fraca para garantir que os lados escuros dos planetas não ficam completamente negros.

4.2. Exemplos de XML Atualizado e Fontes de Texturas

4.2.1. Exemplos de XML

Segue-se um excerto representativo de um XML de teste feito apenas com a Terra contendo várias texturas:

```
<world>
  <window width="1280" height="720" />
  <camera>
    <position x="0" y="60" z="180" />
    <lookAt x="0" y="0" z="0" />
    <up x="0" y="1" z="0" />
    <projection fov="45" near="1" far="2000" />
  </camera>

  <globalAmbient r="0.12" g="0.12" b="0.12" a="1.0" />
```

```

<lights>
  <light type="point" posX="500" posY="0" posZ="0"
    r="255" g="240" b="200" intensity="1.0" />
</lights>

<group>
  <transform>
    <rotate time="40" x="0" y="1" z="0" />
    <scale x="8.0" y="8.0" z="8.0" />
  </transform>
  <models>
    <model file="icosphere.obj" shader="earth_daynight">
      <texture role="diffuse" file="textures/2k_earth_daymap.jpg"
blend="multiply" opacity="1.0" />
      <texture role="night" file="textures/2k_earth_nightmap.jpg"
blend="mix" opacity="0.9" mixFactor="1-nd1" />
      <texture role="clouds" file="textures/2k_earth_clouds.jpg"
blend="overlay" opacity="0.4" />
      <texture role="specular" file="textures/2k_earth_specular_map.jpg"
blend="add" opacity="1.0" useChannel="r" mixFactor="specular" />
      <color>
        <diffuse R="43" G="130" B="201" />
        <ambient R="4" G="13" B="20" />
        <specular R="150" G="160" B="200" />
        <emissive R="0" G="0" B="0" />
        <shininess value="128" />
      </color>
    </model>
  </models>
</group>
</world>

```

A tag <globalAmbient> serve para definir a intensidade da luz ambiente, conferindo maior flexibilidade.

4.2.2. Fontes de Texturas

As texturas utilizadas no modelo do sistema solar foram obtidas do repositório público Solar System Scope Textures [4], que disponibiliza imagens de alta resolução de todos os corpos do sistema solar em formato equirretangular, adequado para mapeamento esférico.

Foram utilizadas as texturas para todos os planetas e da lua da terra deste repositório. Por simplicidade, o restante dos objetos do modelo não lhes foi atribuída uma textura.

4.2.3. Nota sobre o Scatter

Durante o desenvolvimento da fase anterior, foi experimentada a utilização de geometria do tipo *scatter* (distribuição aleatória de pontos sobre uma superfície esférica) para

representar os cinturões de asteroides com instâncias individuais texturadas. No entanto, esta abordagem revelou-se problemática do ponto de vista do mapeamento de textura: a geração de coordenadas UV coerentes para partículas com orientação arbitrária no espaço é um problema de difícil tratamento, uma vez que as normais e as coordenadas de textura dos vértices não têm uma orientação consistente entre instâncias.

4.3. Resultados

Os resultados obtidos demonstram a integração bem-sucedida de iluminação, texturas e shaders no modelo do sistema solar. As figuras seguintes ilustram aspetos representativos da cena final.

A Figura 12 mostra um teste feito com iluminação detalhada.

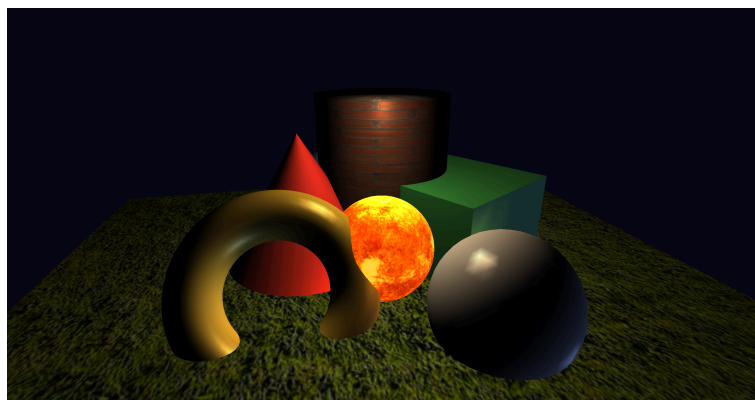


Figura 12: Teste iluminação.

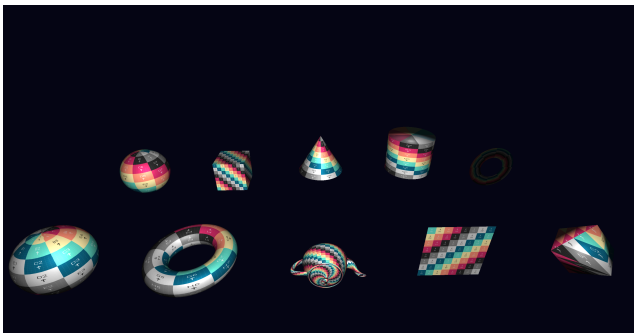


Figura 13: Teste com textura de mapeamento.

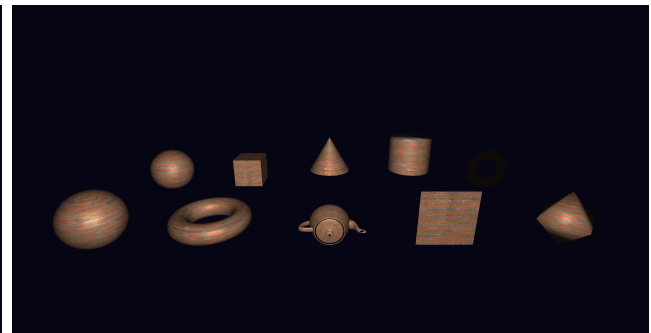


Figura 14: Teste com textura de de tijolo.

A Figura 12 e Figura 13 demonstram um teste onde se dispuseram todas as figuras que o *generator* consegue gerar e se comprovou que o mapeamento estava a ser feito adequadamente através da textura de teste de mapeamento (Figura 15) e uma textura de tijolo.

A0 ↑	A1 ↑	A2 ↑	A3 ↑	A4 ↑	A5 ↑	A6 ↑	A7 ↑
B0 ↑	B1 ↑	B2 ↑	B3 ↑	B4 ↑	B5 ↑	B6 ↑	B7 ↑
C0 ↑	C1 ↑	C2 ↑	C3 ↑	C4 ↑	C5 ↑	C6 ↑	C7 ↑
D0 ↑	D1 ↑	D2 ↑	D3 ↑	D4 ↑	D5 ↑	D6 ↑	D7 ↑
E0 ↑	E1 ↑	E2 ↑	E3 ↑	E4 ↑	E5 ↑	E6 ↑	E7 ↑
F0 ↑	F1 ↑	F2 ↑	F3 ↑	F4 ↑	F5 ↑	F6 ↑	F7 ↑
G0 ↑	G1 ↑	G2 ↑	G3 ↑	G4 ↑	G5 ↑	G6 ↑	G7 ↑
H0 ↑	H1 ↑	H2 ↑	H3 ↑	H4 ↑	H5 ↑	H6 ↑	H7 ↑

Figura 15: Textura mapeamento.

As Figura 15 e Figura 16 representam resultados relativos ao modelo detalhado da Terra que foi desenvolvido, tendo a junção da textura da Terra de dia e de noite, uma componente com um mapa especular e nuvens.

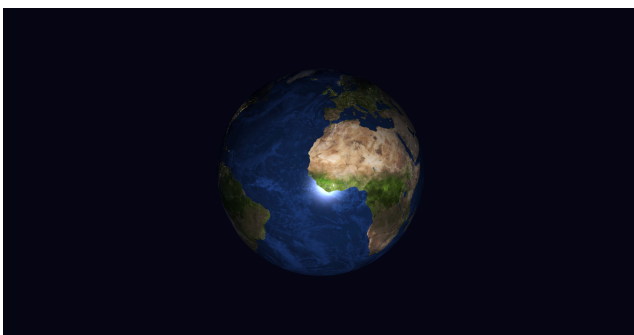


Figura 16: Modelo detalhado da terra (visível componente especular e nuvens).

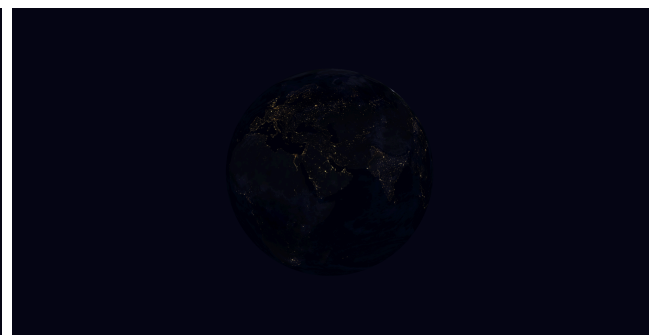


Figura 17: Modelo detalhado da terra (visível componente noturna).

As Figuras abaixo são alguns dos resultados do novo modelo do sistema solar.

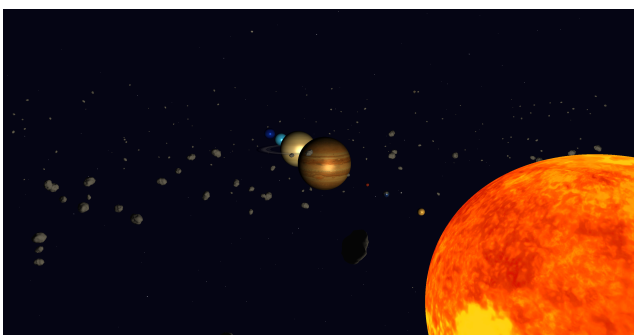


Figura 18: Modelo do sistema solar vista interna.

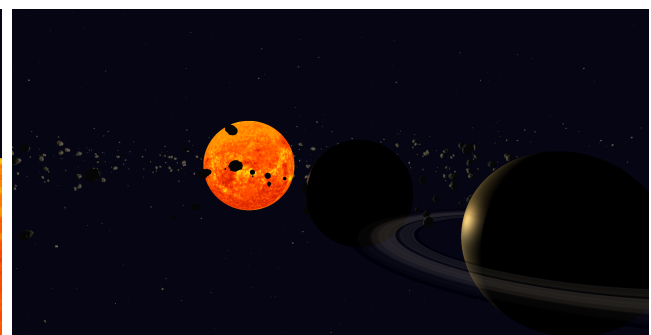


Figura 19: Modelo do sistema solar vista externa.

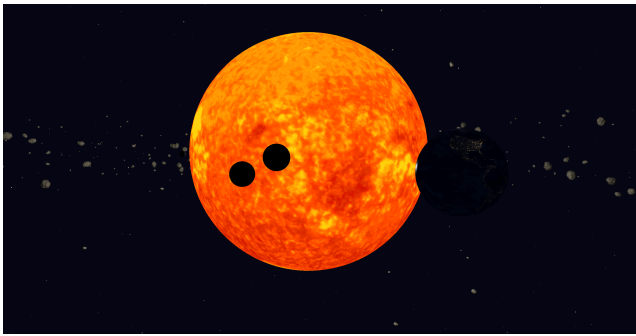


Figura 20: Modelo do sistema solar vista do Sol.

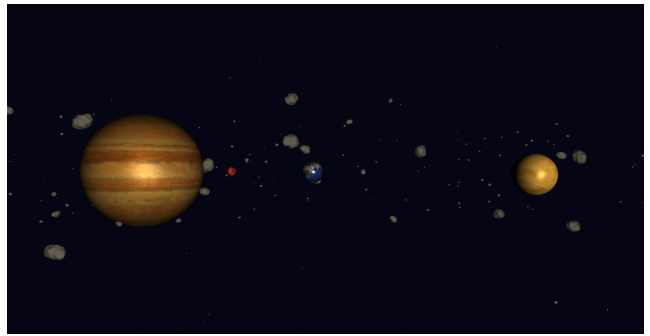


Figura 21: Modelo do sistema solar vista interna dos planetas próximos.

5. Conclusão

A quarta e última fase do trabalho prático foi concluída com sucesso, marcando a finalização do projeto SolariUM — um motor gráfico completo capaz de gerar, animar e renderizar cenas tridimensionais com iluminação, texturas e shaders.

Do lado do *generator*, a migração para o formato `.obj` constituiu uma alteração estrutural importante, que permitiu incorporar normais e coordenadas de textura UV na geometria gerada. A correta computação destas propriedades para cada primitiva — com fórmulas matemáticas adaptadas à geometria de cada figura — foi essencial para a obtenção de resultados visuais coerentes com o modelo de iluminação e o mapeamento de textura.

Do lado da *engine*, foram implementados três sistemas interdependentes: o modelo de iluminação de Phong por fragmento, o carregamento e aplicação de texturas 2D com *mipmapping*, e um sistema de shaders GLSL que centraliza a lógica de renderização nos programas da GPU. A introdução dos shaders foi particularmente significativa, pois conferiu à engine a expressividade necessária para combinar múltiplas camadas de textura de forma fisicamente motivada — como demonstrado pelo shader da Terra, que mistura texturas de dia, noite, nuvens e reflexo especular de acordo com a iluminação incidente em cada fragmento.

O modelo do sistema solar beneficiou de todas estas adições, tornando-se visualmente muito mais rico do que nas fases anteriores. As texturas obtidas com resoluções de alta qualidade e o shader especializado da Terra foram os elementos que mais contribuíram para o realismo da cena final.

Numa perspetiva global, o projeto SolariUM cumpriu integralmente os objetivos curriculares das quatro fases, evoluindo progressivamente até um motor gráfico com *pipeline* moderna baseada em VBOs, VAOs, shaders GLSL, animações procedimentais e renderização multi-textura. As escolhas de implementação — nomeadamente a adoção de VAOs desde a fase três e a introdução de *shaders* nesta fase — posicionam o código numa arquitetura moderna e extensível, próxima das práticas utilizadas na indústria de desenvolvimento de aplicações gráficas em tempo real.

Bibliografia

- [1] Khronos Group, «OpenGL - The Industry Standard for High Performance Graphics». [Em linha]. Disponível em: <https://www.opengl.org/>
- [2] Library of Congress, «Sustainability of Digital Formats: Planning for Library of Congress Collections». [Em linha]. Disponível em: <https://www.loc.gov/preservation/digital/formats/fdd/fdd000507.shtml>
- [3] B. Dorta, «Phong Model». [Em linha]. Disponível em: <https://www.brunodorta.com.br/cg/phongmodel>
- [4] Solar System Scope, «Solar System Scope Textures». [Em linha]. Disponível em: <https://www.solarsystemscope.com/textures>

Lista de Siglas e Acrónimos

API Application Programming Interface,
CCW Counter-Clockwise,
CPU Central Processing Unit,
FOV Field of View,
GLSL OpenGL Shading Language,
GPU Graphics Processing Unit,
UV Coordenadas de textura (U, V),
VAO Vertex Array Object,
VBO Vertex Buffer Object,
XML Extensible Markup Language,

Anexos

Anexo 1: Logo da Universidade do Minho

